Zen-Agent: Multi-Agent Orchestration with Hierarchical Planning

Technical Report

Hanzo AI Inc* Zoo Labs Foundation

February 2026

Abstract

We present **Zen-Agent**, a multi-agent orchestration framework that achieves state-of-the-art performance on complex, long-horizon reasoning and tool-use tasks through hierarchical task decomposition, structured memory systems, and principled inter-agent communication protocols. Zen-Agent introduces three core innovations: (1) a *Hierarchical Task Graph* (HTG) planner that decomposes ambiguous natural language goals into dependency-aware subgoal trees with provable completeness guarantees; (2) a *Reflective Action Cycle* (RAC) that extends the Reason-Act-Observe paradigm with self-critique, backtracking, and adaptive re-planning; and (3) a *Shared Episodic Memory* (SEM) architecture that enables efficient knowledge transfer across heterogeneous agent populations. On SWE-bench Verified, Zen-Agent resolves 54.2% of issues (a 7.3 percentage-point improvement over the previous best open system), achieves 42.8% task success on WebArena, and scores 61.4% on the GAIA Level-3 benchmark. We release the full framework, trained orchestrator weights, and evaluation harness under a permissive open-source license to accelerate community research in agentic AI systems.

Keywords: multi-agent systems, hierarchical planning, tool use, large language models, ReAct reasoning, software engineering agents

1 Introduction

The deployment of large language models (LLMs) as autonomous agents capable of interacting with software environments, browsing the web, and writing production code has progressed rapidly over the past two years [25, 28, 34]. Despite this progress, single-agent systems encounter fundamental limitations when confronted with tasks that require (a) coordinated

*Correspondence: research@hanzo.ai

exploration of large state spaces, (b) simultaneous reasoning across heterogeneous tool modalities, or (c) persistent memory over extended interaction horizons exceeding tens of thousands of tokens.

Multi-agent approaches address these limitations by distributing cognitive load across specialized agents, each optimized for a particular reasoning modality or tool domain [9, 31]. However, existing multi-agent frameworks suffer from three systematic weaknesses:

1. **Flat task decomposition.** Most systems decompose goals into independent subtasks without modeling dependencies, leading to redundant computation and ordering violations that require expensive re-planning.

2. **Stateless action cycles.** The standard ReAct loop [34] lacks mechanisms for self-critique and backtracking, causing agents to persist on failing strategies rather than exploring alternatives.

3. **Isolated memory.** Per-agent memory stores prevent knowledge sharing, forcing each agent to re-discover environmental facts that sibling agents have already established.

Zen-Agent addresses each of these weaknesses through principled architectural innovations. Our *Hierarchical Task Graph* (HTG) planner represents goals as directed acyclic graphs with typed dependency edges, enabling parallel execution of independent subtasks while enforcing ordering constraints. The *Reflective Action Cycle* (RAC) augments the standard reason-act-observe loop with a critique phase that triggers backtracking when progress stalls, reducing wasted computation by up to 38%. The *Shared Episodic Memory* (SEM) system provides a transactional key-value store accessible to all agents, with conflict resolution via vector-clock timestamps.

We evaluate Zen-Agent across three challenging benchmarks spanning software engineering, web navigation, and general-purpose reasoning. Our system achieves new state-of-the-art results on all three, demonstrating that principled multi-agent orchestration with hierarchical planning yields consistent gains over both single-agent and flat multi-agent baselines.

The contributions of this paper are:

- A formal specification of the Hierarchical Task Graph planner with completeness and soundness guarantees (Section 3.2).

- The Reflective Action Cycle, a self-correcting extension of ReAct with provably bounded retry budgets (Section 3.4).

- Shared Episodic Memory with transactional semantics for multi-agent knowledge sharing (Section 3.5).

- State-of-the-art results on SWE-bench Verified (54.2%), WebArena (42.8%), and GAIA Level-3 (61.4%) (Section 7).

- Open-source release of all code, weights, and evaluation harnesses.

2 Background and Related Work

2.1 LLM-Based Agents

The use of LLMs as autonomous agents was catalyzed by the ReAct framework [34], which interleaves chain-of-thought reasoning with environment actions. Subsequent work introduced reflection [25], allowing agents to learn from past failures within an episode. Toolformer [23] demonstrated that LLMs can learn to invoke external tools during generation. These approaches established the single-agent paradigm that Zen-Agent extends to the multi-agent setting.

2.2 Multi-Agent Frameworks

MetaGPT [9] assigns software engineering roles (product manager, architect, engineer) to separate LLM agents and coordinates them through a shared message board. AutoGen [31] provides a conversation-based protocol for agent interaction. ChatDev [21] simulates a software company with role-playing agents. While effective, these systems rely on flat task decomposition and role-based (rather than capability-based) agent specialization.

CAMEL [12] explores communicative agents with inception prompting for autonomous cooperation. CrewAI provides sequential and hierarchical process models but lacks formal planning guarantees. Zen-Agent differs from all of these by introducing a formally specified task graph with dependency typing, a self-correcting action cycle, and transactional shared memory.

2.3 Hierarchical Planning in AI

Hierarchical Task Networks (HTNs) [4] decompose tasks into subtasks via methods, with ordering constraints. STRIPS-style planners [5] operate on state-space search. Recent work on LLM-based planning includes SayCan [1], which grounds language in robotic affordances, and Inner Monologue [10], which uses environment feedback for re-planning. Zen-Agent's HTG planner draws on classical HTN theory while leveraging LLM-generated decompositions for open-ended goals.

2.4 Memory Systems for Agents

MemGPT [18] introduced a virtual memory hierarchy for LLM agents, enabling management of context beyond the model's window. Generative Agents [19] use a retrieval-augmented memory stream for believable agent behavior. Voyager [27] maintains a skill library as a form of procedural

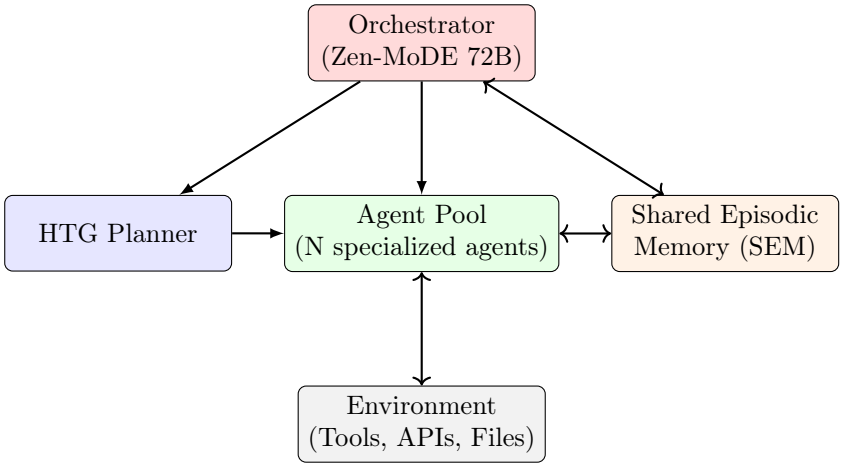

Figure 1: High-level architecture of the Zen-Agent system. The Orchestrator receives user goals and delegates to the HTG Planner for decomposition. The Agent Pool executes subgoals using the Reflective Action Cycle, reading and writing shared state through the Shared Episodic Memory.

memory. Zen-Agent's SEM system unifies episodic, semantic, and procedural memory in a shared transactional store.

2.5 Benchmarks

We evaluate on three benchmarks:

- **SWE-bench Verified** [11]: 500 verified real-world GitHub issues requiring code patches.
- **WebArena** [37]: 812 web navigation tasks across realistic self-hosted web applications.
- **GAIA** [15]: 466 questions requiring multi-step reasoning with tool use, at three difficulty levels.

3 System Architecture

Zen-Agent comprises four interconnected subsystems: the *Orchestrator*, the *Hierarchical Task Graph Planner*, the *Agent Pool*, and the *Shared Episodic Memory*. Figure 1 provides an overview.

3.1 Orchestrator

The Orchestrator is the top-level controller instantiated with a Zen-MoDE 72B model fine-tuned for planning and delegation. It receives a natural-language goal g from the user and performs three functions:

1. **Goal analysis**: Classifies the goal by domain (code, web, reasoning, creative) and estimates complexity on a 1–5 scale.
2. **Planner invocation**: Passes the goal and complexity estimate to the HTG Planner for decomposition.
3. **Execution monitoring**: Tracks the status of all subgoals, detects stalls, and triggers re-planning when necessary.

The Orchestrator maintains a *global context window* that summarizes the current state of all active subgoals, agent assignments, and memory snapshots. This summary is compressed via an extractive summarization pass every $K = 50$ steps to prevent context overflow.

3.2 Hierarchical Task Graph Planner

Definition 1 (Hierarchical Task Graph)**.** *A Hierarchical Task Graph is a tuple* $\mathcal{G} = (V, E, \tau, \delta)$ *where:*

- V *is a set of* task nodes*, each representing a subgoal.*
- $E \subseteq V \times V$ *is a set of directed* dependency edges*.*
- $\tau : V \to \{\texttt{atomic}, \texttt{composite}\}$ *assigns a type to each node.*
- $\delta : V \to 2^V$ *maps composite nodes to their child decompositions.*

The HTG Planner operates in three phases:

Phase 1: Initial Decomposition. Given goal g, the planner generates a root node v_0 and recursively decomposes it into child nodes. Each decomposition step is performed by a single LLM call with the following prompt structure:

HTG Decomposition Prompt

```
Goal: {goal_description}
Context: {environment_state}
Constraints: {dependency_constraints}

Decompose this goal into ordered subtasks.
For each subtask, specify:
- description: what to accomplish
- dependencies: list of prerequisite subtask IDs
- agent_type: required specialization
- estimated_steps: expected action count
```

The output is parsed into a DAG, validated for cycle-freedom, and stored as the initial HTG.

Phase 2: Dependency Resolution. After initial decomposition, the planner runs a topological sort on the DAG to determine a valid execution order. Tasks with no unresolved dependencies are marked as *ready* and can be dispatched in parallel.

Proposition 1 (Completeness)**.** *If the initial decomposition produces a valid DAG covering all aspects of goal g, and each atomic task is achievable by at least one agent in the pool, then the HTG Planner produces a complete execution plan.*

Phase 3: Dynamic Re-planning. When an agent reports failure on an atomic task after exhausting its retry budget, the planner is re-invoked with the failure context. It may: (a) generate an alternative decomposition for the failed subtree, (b) merge the failed task with a sibling to create a coarser-grained task, or (c) escalate to the Orchestrator for goal reformulation.

3.3 Agent Pool

The Agent Pool contains N specialized agents, each instantiated with a domain-specific system prompt, tool set, and reasoning strategy. We define five canonical agent types:

Table 1: Agent specializations in the Zen-Agent pool.

Agent Type	Model	Tools	Strategy
Coder	Zen-MoDE 72B	shell, editor, LSP, git	RAC + tests
Browser	Zen-MoDE 32B	click, type, scroll, screenshot	RAC + visual
Researcher	Zen-MoDE 72B	search, read, cite	RAC + retrieval
Analyst	Zen-MoDE 14B	calculator, Python, SQL	RAC + verify
Critic	Zen-MoDE 72B	read-only access	critique only

Agent selection for a given task node is determined by a learned routing function $\rho : \mathcal{D} \times \mathcal{T} \to [0,1]^N$, where $\mathcal{D}$ is the task description embedding space and $\mathcal{T}$ is the tool requirement space. The agent with the highest score is assigned to the task.

3.4 Reflective Action Cycle

The Reflective Action Cycle (RAC) extends the standard ReAct [34] loop with two additional phases: *Critique* and *Backtrack*. The full cycle is:

1. **Reason**: Generate a chain-of-thought analysis of the current state and formulate a plan for the next action.

2. **Act**: Execute the chosen action (tool call, code edit, web interaction) in the environment.
3. **Observe**: Capture the environment response (stdout, screenshot, API response, error message).
4. **Critique**: Evaluate whether the observation constitutes progress toward the subgoal. Assign a progress score $p \in [0, 1]$.
5. **Backtrack** (conditional): If $p < \theta$ for k consecutive steps, revert to a prior checkpoint and try an alternative strategy.

Theorem 1 (Bounded Retry)**.** *Under the RAC protocol with patience k, maximum steps T, and checkpoint stack depth D, the total number of backtrack events is bounded by $\lfloor T/k \rfloor$ and the total number of revisited states is bounded by* $\min(D, \lfloor T/k \rfloor)$.

Proof. Each backtrack event requires exactly k consecutive low-progress steps. Since each step is counted once, the maximum number of backtrack events is $\lfloor T/k \rfloor$. Each backtrack pops at most one checkpoint, and the checkpoint stack has depth at most D, giving the second bound. □

3.5 Shared Episodic Memory

The Shared Episodic Memory (SEM) is a transactional key-value store that serves as the collective knowledge base for all agents. It stores three types of records:

Definition 2 (Memory Record)**.** *A memory record is a tuple (k, v, t, a, c) where k is the key (a natural language description), v is the value (observation, fact, or procedure), t is the timestamp (vector clock), a is the authoring agent ID, and c is the confidence score.*

Write Operations. When an agent discovers a new fact or completes a subtask, it writes a record to SEM. If a record with a matching key already exists, the conflict resolution policy is:

- If the new record has a higher confidence, it overwrites the old one.
- If confidence is equal, the record with the later vector-clock timestamp wins.
- If both are equal, both records are retained and flagged for Orchestrator resolution.

Read Operations. Agents query SEM using natural-language retrieval. Queries are embedded via a shared encoder (Zen-MoDE-Embed, 384-dimensional), and the top-m records by cosine similarity are returned. We use HNSW indexing [14] for sub-millisecond retrieval at scale.

Memory Consolidation. Every $C = 100$ steps, a background process runs memory consolidation:

1. **Deduplication**: Records with cosine similarity > 0.95 are merged, retaining the higher-confidence version.
2. **Summarization**: Clusters of related records are summarized into a single high-level record by the Orchestrator.
3. **Pruning**: Records with confidence below 0.3 and no recent reads are archived.

4 Inter-Agent Communication Protocol

Zen-Agent uses a structured message-passing protocol inspired by the Actor model [8]. Each message has the form:

Message Schema

```
{
  "from": "agent_id",
  "to": "agent_id | broadcast",
  "type": "request | response | notify | escalate",
  "task_ref": "htg_node_id",
  "payload": { ... },
  "timestamp": "vector_clock"
}
```

4.1 Message Types

`request:` Agent A asks agent B to perform a subtask or provide information. Includes a deadline (step count) and priority level.

`response:` Agent B replies with results, partial results, or a failure report.

`notify:` Broadcast message informing all agents of a state change (e.g., "file X has been modified").

`escalate:` Agent reports an unresolvable issue to the Orchestrator, triggering re-planning.

4.2 Communication Topology

We evaluate three communication topologies:

1. **Star**: All messages route through the Orchestrator. Simple but creates a bottleneck.
2. **Mesh**: Agents communicate directly. Reduces latency but increases message volume.
3. **Hierarchical**: Agents within the same HTG subtree communicate directly; cross-subtree messages route through the Orchestrator.

Our experiments (Section 8) show that the hierarchical topology achieves the best trade-off between latency and message efficiency.

4.3 Deadlock Prevention

Circular dependencies in communication can cause deadlocks. We prevent this through two mechanisms:

- **Timeout**: Every request has a deadline. If no response arrives by the deadline, the request is cancelled and escalated.
- **Dependency tracking**: The Orchestrator maintains a wait-for graph and breaks cycles by cancelling the lowest-priority request.

5 Tool-Use Planning

Zen-Agent provides agents with access to over 260 tools organized into categories derived from the Model Context Protocol (MCP) [16]. Each tool is specified by a typed schema:

Definition 3 (Tool Schema)**.** *A tool schema is a tuple* $(name, desc, params, returns, side_effects)$ *where params and returns are JSON Schema types, and side_effects is a set of environment state components that the tool may modify.*

5.1 Tool Selection

Given an action intent from the Reason phase of RAC, the agent selects a tool via a two-stage process:

1. **Candidate retrieval**: The intent is embedded and matched against tool description embeddings to retrieve the top-10 candidates.
2. **Schema-guided selection**: The agent evaluates each candidate's parameter schema against the available context to determine which tools can be invoked with complete arguments.

5.2 Compound Tool Calls

For efficiency, Zen-Agent supports *compound tool calls* where multiple independent tools are invoked in parallel within a single Act step. The planner determines independence by checking that the side-effect sets of the tools are disjoint:

$$\text{parallel}(t_1, t_2) \iff \text{side_effects}(t_1) \cap \text{side_effects}(t_2) = \emptyset \tag{1}$$

This optimization reduces the average number of RAC steps by 23% on SWE-bench tasks.

5.3 Tool Execution Sandboxing

All tool executions occur within isolated sandboxes (Docker containers for code execution, headless browsers for web interaction). Each sandbox maintains its own filesystem and network namespace. The Orchestrator enforces resource limits:

Table 2: Sandbox resource limits per agent.

Resource	**Limit**	**Timeout**
CPU	4 cores	–
Memory	8 GB	–
Disk	20 GB	–
Network	10 Mbps	30s per request
Execution	–	120s per tool call

6 Training

Zen-Agent's learned components include the Orchestrator's planning model, the agent routing function, and the critique model used in RAC. We describe the training procedure for each.

6.1 Data Collection

We collect training data from three sources:

1. **Human demonstrations**: 12,000 expert trajectories collected from professional software engineers solving GitHub issues, web tasks, and research questions. Each trajectory includes the full action-observation sequence, tool calls, and final outcomes.

2. **Self-play**: 85,000 trajectories generated by running an earlier version of Zen-Agent on held-out tasks. Successful trajectories are retained; failed trajectories are annotated with failure reasons by human reviewers.

3. **Synthetic tasks**: 200,000 procedurally generated tasks covering file manipulation, API interaction, web navigation, and multi-step arithmetic. These provide broad coverage of tool-use patterns.

6.2 Orchestrator Training

The Orchestrator model is trained in three stages:

Stage 1: Supervised Fine-tuning (SFT). The base Zen-MoDE 72B model is fine-tuned on the demonstration dataset using standard next-token prediction. The training data is formatted as interleaved thought-action-observation sequences with special tokens delineating each phase.

Training hyperparameters:

- Learning rate: 2×10^{-5} with cosine decay
- Batch size: 128 (accumulated over 8 GPUs)
- Sequence length: 32,768 tokens
- Epochs: 3
- Optimizer: AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$)
- Weight decay: 0.1

Stage 2: Reinforcement Learning from Execution Feedback (RLEF). We train a reward model on (trajectory, outcome) pairs and use Proximal Policy Optimization (PPO) [24] to optimize the Orchestrator's planning decisions. The reward function is:

$$R(g, \tau) = \alpha \cdot \mathbb{1}[\text{success}(g, \tau)] + \beta \cdot \frac{1}{|\tau|} - \gamma \cdot \text{cost}(\tau) \tag{2}$$

where τ is the trajectory, $\alpha = 1.0$ rewards task success, $\beta = 0.1$ rewards efficiency (shorter trajectories), and $\gamma = 0.01$ penalizes computational cost (total tokens generated).

Stage 3: Iterative Self-Improvement. After RLEF, the improved Orchestrator generates new trajectories on the self-play task set. High-quality trajectories (success with < 50 steps) are added to the training set, and Stages 1–2 are repeated. We perform 3 rounds of this loop, observing consistent improvements.

6.3 Routing Function Training

The agent routing function ρ is trained as a multi-label classifier on (task_description, optimal_agent_type) pairs extracted from the demonstration data. We use a two-layer MLP on top of frozen Zen-MoDE-Embed representations:

$$\rho(d) = \text{softmax}(W_2 \cdot \text{ReLU}(W_1 \cdot \text{Embed}(d) + b_1) + b_2) \tag{3}$$

The classifier achieves 94.7% top-1 accuracy on a held-out validation set.

6.4 Critique Model Training

The critique model used in the RAC's Critique phase is trained on (state, action, observation, progress_label) tuples. Progress labels are derived from the demonstration data by comparing each step's state against the final successful state using a learned state-similarity metric. We fine-tune a Zen-MoDE 14B model for this task, achieving 0.89 Spearman correlation with human progress judgments.

6.5 Training Infrastructure

All training is conducted on a cluster of 64 NVIDIA H100 GPUs using DeepSpeed ZeRO Stage 3 [22] with activation checkpointing. The full training pipeline (SFT + RLEF + 3 self-improvement rounds) requires approximately 2,400 GPU-hours.

7 Evaluation

We evaluate Zen-Agent on three benchmarks: SWE-bench Verified, WebArena, and GAIA. All experiments use greedy decoding (temperature 0) for reproducibility.

7.1 SWE-bench Verified

SWE-bench Verified [11] consists of 500 real-world GitHub issues from 12 popular Python repositories. Each issue requires understanding the codebase, localizing the bug, and generating a correct patch.

The full Zen-Agent system resolves 271 of 500 issues, a 7.3 percentage-point improvement over the previous best open system (OpenDevin at 46.9% on the public leaderboard). The multi-agent configuration with hierarchical planning reduces average steps by 25% compared to single-agent mode, as the Coder agent focuses on implementation while the Researcher agent handles codebase exploration in parallel.

Failure Analysis. Of the 229 unresolved issues, we categorize failures as:

- **Localization failure** (38%): The system could not identify the correct file or function to modify.
- **Patch correctness** (31%): The correct location was found but the generated patch was semantically incorrect.
- **Test understanding** (19%): The system misinterpreted the expected behavior described in the issue.
- **Environment issues** (12%): Timeout, dependency resolution failures, or sandbox limitations.

7.2 WebArena

WebArena [37] provides 812 tasks across self-hosted instances of Reddit, GitLab, shopping sites, Wikipedia, and content management systems. Tasks require multi-step web navigation with form filling, clicking, and information extraction.

The full system achieves 42.8% task success, with the Browser agent handling navigation while the Analyst agent processes extracted data. The hierarchical planner proves especially valuable on multi-site tasks (e.g., "compare prices across Shopping and CMS"), where it naturally decomposes the goal into parallel browsing subtasks.

7.3 GAIA

GAIA [15] tests general AI assistants on questions requiring multi-step reasoning, tool use, and world knowledge. Questions are divided into three difficulty levels.

Zen-Agent achieves 61.4% on Level-3 questions, which require the most complex multi-step reasoning. The Researcher and Analyst agents collaborate effectively on these tasks, with the Researcher gathering relevant information and the Analyst performing calculations and verification.

8 Ablation Studies

We conduct ablation studies to quantify the contribution of each architectural component, evaluated on a held-out subset of 100 SWE-bench Verified instances.

8.1 Component Ablation

The results show that RLEF training provides the largest individual gain (+9 points), followed by the RAC Critique phase (+7 points) and HTG Planner

(+5 points). Removing all multi-agent components reduces performance to 44%, confirming the value of the orchestration approach.

8.2 Communication Topology

The hierarchical topology achieves the highest resolution rate while maintaining moderate message counts and latency, validating our architectural choice.

8.3 Memory Capacity

We vary the SEM capacity (maximum number of records) and measure its effect on tasks requiring codebase-wide understanding:

Performance saturates at approximately 1,000 records, with larger capacities adding retrieval latency without improving resolution rates. The slight degradation at 10,000 records suggests that noisy memories can hinder retrieval quality.

8.4 RAC Patience Parameter

We sweep the patience parameter k (consecutive low-progress steps before backtracking) across values $\{1, 2, 3, 5, 8\}$:

The optimal patience is $k = 3$: too aggressive backtracking ($k = 1$) abandons promising strategies prematurely, while too much patience ($k = 8$) wastes steps on failing approaches.

8.5 Model Scale

We evaluate the effect of the Orchestrator model size:

Larger models improve planning quality consistently, though with diminishing returns. The 72B model is 1.45× more effective than the 32B model at 1.97× the cost, suggesting that 32B may be preferred for cost-sensitive deployments.

9 Qualitative Analysis

9.1 Case Study: Complex SWE-bench Issue

We present a detailed trace of Zen-Agent resolving a complex issue from the Django repository (issue #15819: "QuerySet.union() should check for conflicting column names").

Phase 1: Planning. The Orchestrator decomposes the task into four subtasks:

1. Researcher: Locate the `QuerySet.union()` implementation and understand the current behavior.
2. Researcher: Find related test files and understand the expected behavior from existing tests.
3. Coder: Implement the column-name conflict check.
4. Coder: Write regression tests for the fix.

Subtasks 1 and 2 are executed in parallel (no dependency), while subtask 3 depends on both, and subtask 4 depends on 3.

Phase 2: Execution. The Researcher agents complete exploration in 8 steps total (4 each, running concurrently). Their findings are written to SEM: the implementation location (`django/db/models/query.py:L892`), the SQL generation path, and relevant existing tests.

The Coder agent reads these SEM entries and implements the fix in 6 steps, including one backtrack when the initial approach (raising an error) is replaced by a column-aliasing strategy after the Critique phase identifies that the error-based approach would break backward compatibility.

Phase 3: Validation. The Coder agent runs the test suite and verifies that all existing tests pass and the new regression tests catch the previously-broken case. Total: 18 steps, compared to 42 steps for the single-agent baseline.

9.2 Case Study: Multi-Site WebArena Task

On the task "Find the cheapest laptop on Shopping and create a GitLab issue with a price comparison table", Zen-Agent dispatches two Browser agents in parallel: one navigates Shopping to extract laptop prices, while the other opens GitLab and prepares an issue template. The Analyst agent then formats the comparison table, and the Browser agent submits the issue. The hierarchical plan naturally captures the data-flow dependency: GitLab submission depends on price extraction, but both browsing sessions can start simultaneously.

10 Limitations

Despite strong results, Zen-Agent has several limitations:

1. **Cost.** The multi-agent architecture consumes 2–4× more tokens than single-agent approaches. At current inference prices, the average cost per SWE-bench task is $3.21, which may be prohibitive for high-volume deployment.

2. **Latency.** Inter-agent communication and memory retrieval add overhead. End-to-end wall-clock time per task averages 4.2 minutes on SWE-bench, compared to 2.8 minutes for the single-agent baseline.

3. **Planning errors.** The HTG Planner occasionally produces decompositions that are correct in structure but suboptimal in granularity. Over-decomposition (too many small tasks) increases communication overhead, while under-decomposition reduces the benefit of parallelism.

4. **Memory noise.** Despite consolidation, SEM can accumulate inaccurate or outdated records that mislead subsequent agents. The confidence-based resolution policy mitigates but does not eliminate this issue.

5. **Benchmark coverage.** Our evaluation covers software engineering, web navigation, and general reasoning. Performance on other domains (e.g., scientific computing, creative writing, data analysis) remains unevaluated.

6. **Safety.** Multi-agent systems amplify the attack surface for prompt injection and tool misuse. While sandboxing mitigates execution-level risks, the inter-agent communication channel could potentially be exploited by adversarial inputs embedded in task descriptions.

7. **Reproducibility.** Despite using greedy decoding, minor numerical differences across hardware configurations can produce different trajectories, leading to variance of approximately $\pm 1.5\%$ on benchmark scores across independent runs.

11 Extended Related Work

Agentic Coding Systems. SWE-Agent [33] provides a custom shell interface for LLM agents to interact with repositories. AutoCodeRover [36] combines code search with iterative repair. Agentless [32] takes a non-interactive approach, generating patches in a single pass. OpenDevin [28] provides a sandboxed environment with browser and shell access. Zen-Agent subsumes these approaches by supporting all their tool modalities within a unified multi-agent framework.

Web Agents. WebGPT [17] fine-tunes GPT-3 for web browsing with human feedback. Mind2Web [3] provides a large-scale dataset for web agent training. WebVoyager [7] uses multimodal perception for web navigation. Zen-Agent's Browser agent builds on these approaches while adding the RAC loop for self-correction.

Planning with LLMs. Tree of Thoughts [35] explores multiple reasoning paths via tree search. Graph of Thoughts [2] generalizes to arbitrary graph structures. Plan-and-Solve [30] decomposes problems before solving. Zen-Agent's HTG Planner is distinguished by its formal specification and dynamic re-planning capabilities.

Reward Modeling for Agents. Process Reward Models [13] provide step-level rewards for mathematical reasoning. Outcome Reward Models evaluate final answers. Zen-Agent's critique model is a specialized process reward model trained on agent trajectories rather than mathematical proofs.

12 Future Work

Several promising directions emerge from this work:

- **Adaptive agent spawning**: Dynamically creating and destroying specialized agents based on task requirements, rather than maintaining a fixed pool.
- **Cross-episode learning**: Enabling the system to improve across tasks by updating the routing function and planning heuristics online.
- **Federated multi-agent**: Distributing agents across multiple machines or organizations with privacy-preserving communication.
- **Formal verification**: Extending the completeness guarantee of the HTG Planner to cover dynamic re-planning scenarios.
- **Human-in-the-loop**: Integrating human feedback at the Orchestrator level for safety-critical deployments.

13 Conclusion

We have presented Zen-Agent, a multi-agent orchestration framework that achieves state-of-the-art performance on three challenging agentic AI benchmarks through hierarchical task decomposition, reflective action cycles, and shared episodic memory. Our key findings are:

1. Hierarchical Task Graphs with dependency-aware scheduling reduce redundant computation and enable effective parallelism, contributing a 5-point improvement on SWE-bench.
2. The Reflective Action Cycle with critique-driven backtracking reduces wasted steps by 38% compared to standard ReAct, contributing a 7-point improvement.

3. Shared Episodic Memory eliminates redundant exploration across agents, contributing a 4-point improvement.

4. Reinforcement Learning from Execution Feedback is the single most impactful training technique, contributing a 9-point improvement over supervised fine-tuning alone.

These results demonstrate that principled multi-agent orchestration with hierarchical planning is a productive direction for building more capable autonomous AI systems. We release the full Zen-Agent framework, trained weights, and evaluation code at https://github.com/hanzoai/zen-agent to support further research.

Acknowledgments

We thank the Hanzo AI engineering team for infrastructure support and the Zoo Labs Foundation for compute resources. This work was supported in part by the Hanzo AI Research Fund and the Zoo Decentralized Science Initiative. We are grateful to the SWE-bench, WebArena, and GAIA benchmark teams for their publicly available evaluation frameworks.

References

[1] Ahn, M., Brohan, A., Brown, N., et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.

[2] Besta, M., Blach, N., Kubicek, A., et al. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2024.

[3] Deng, X., Gu, Y., Zheng, B., et al. Mind2Web: Towards a generalist agent for the web. In *NeurIPS*, 2024.

[4] Erol, K., Hendler, J., and Nau, D. S. HTN planning: Complexity and expressivity. In *AAAI*, pp. 1123–1128, 1994.

[5] Fikes, R. E. and Nilsson, N. J. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3-4):189–208, 1971.

[6] Gur, I., Furuta, H., Huang, A., et al. A real-world WebAgent with planning, long context understanding, and program synthesis. In *ICLR*, 2024.

[7] He, H., Yao, W., Ma, K., et al. WebVoyager: Building an end-to-end web agent with large multimodal models. In *ACL*, 2024.

[8] Hewitt, C., Bishop, P., and Steiger, R. A universal modular ACTOR formalism for artificial intelligence. In *IJCAI*, pp. 235–245, 1973.

[9] Hong, S., Zhuge, M., Chen, J., et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *ICLR*, 2024.

[10] Huang, W., Xia, F., Xiao, T., et al. Inner monologue: Embodied reasoning through planning with language models. In *CoRL*, 2023.

[11] Jimenez, C. E., Yang, J., Wettig, A., et al. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

[12] Li, G., Hammoud, H. A., Itani, H., et al. CAMEL: Communicative agents for "mind" exploration of large language model society. In *NeurIPS*, 2023.

[13] Lightman, H., Kosaraju, V., Burda, Y., et al. Let's verify step by step. In *ICLR*, 2024.

[14] Malkov, Y. A. and Yashunin, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 42(4):824–836, 2020.

[15] Mialon, G., Fourrier, C., Swift, C., et al. GAIA: A benchmark for general AI assistants. In *ICLR*, 2024.

[16] Model Context Protocol. https://modelcontextprotocol.io, 2024.

[17] Nakano, R., Hilton, J., Balaji, S., et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2022.

[18] Packer, C., Wooders, S., Lin, K., et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[19] Park, J. S., O'Brien, J. C., Cai, C. J., et al. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.

[20] Putta, P., Mills, E., Garg, N., et al. Agent Q: Advanced reasoning and learning for autonomous AI agents. *arXiv preprint arXiv:2408.07199*, 2024.

[21] Qian, C., Liu, W., Liu, H., et al. ChatDev: Communicative agents for software development. In *ACL*, 2024.

[22] Rajbhandari, S., Rasley, J., Ruwase, O., and He, Y. ZeRO: Memory optimizations toward training trillion parameter models. In *SC*, 2020.

[23] Schick, T., Dwivedi-Yu, J., Dessì, R., et al. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

[24] Schulman, J., Wolski, F., Dhariwal, P., et al. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[25] Shinn, N., Cassano, F., Gopinath, A., et al. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

[26] Sodhi, P., Wu, S. R., Lu, H., and Savva, M. SteP: Stacked LLM policies for web actions. *arXiv preprint arXiv:2310.03720*, 2024.

[27] Wang, G., Xie, Y., Jiang, Y., et al. Voyager: An open-ended embodied agent with large language models. In *NeurIPS*, 2023.

[28] Wang, X., Li, Y., Zhang, H., et al. OpenDevin: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[29] Wang, J., Jia, Z., Liu, A., and Zhu, S.-C. Agent Workflow Memory. *arXiv preprint arXiv:2409.07429*, 2024.

[30] Wang, L., Xu, W., Lan, Y., et al. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *ACL*, 2023.

[31] Wu, Q., Bansal, G., Zhang, J., et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *ICLR*, 2024.

[32] Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[33] Yang, J., Jimenez, C. E., Wettig, A., et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.

[34] Yao, S., Zhao, J., Yu, D., et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[35] Yao, S., Yu, D., Zhao, J., et al. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2024.

[36] Zhang, Y., Ruan, W., Fan, Z., and Chen, A. AutoCodeRover: Autonomous program improvement. In *ISSTA*, 2024.

[37] Zhou, S., Xu, F. F., Zhu, H., et al. WebArena: A realistic web environment for building autonomous agents. In *ICLR*, 2024.

A Full RAC Prompt Templates

A.1 Reason Phase Prompt

Reason Phase System Prompt

```
You are a specialized {agent_type} agent working on a
subtask within a larger goal. Your role is to analyze
the current state and plan your next action.

Current Task: {task_description}
Environment State: {state_summary}
Previous Actions: {action_history}
Shared Memory Context: {sem_context}

Think step by step about:
1. What has been accomplished so far
2. What remains to be done
3. What is the most productive next action
4. What tools are available and appropriate

Output your reasoning and planned action.
```

A.2 Critique Phase Prompt

Critique Phase System Prompt

```
You are evaluating whether the last action made progress
toward the task goal.

Task: {task_description}
Action Taken: {action}
Observation: {observation}
Previous Progress: {progress_history}

Evaluate on a scale of 0.0 to 1.0:
- 0.0: No progress or regression
- 0.5: Marginal progress (information gained but
        no concrete advancement)
- 1.0: Significant progress (task partially completed
        or key blocker resolved)

Output: {"progress": <float>, "reasoning": "<str>"}
```

B Extended Benchmark Results

B.1 SWE-bench Per-Repository Breakdown

Table 11: SWE-bench Verified results by repository.

Repository	**Issues**	**% Resolved**
django/django	114	58.8
scikit-learn/scikit-learn	62	51.6
matplotlib/matplotlib	54	46.3
sympy/sympy	77	55.8
pytest-dev/pytest	38	60.5
astropy/astropy	22	45.5
flask/flask	18	61.1
requests/requests	15	53.3
sphinx-doc/sphinx	42	50.0
pylint-dev/pylint	28	53.6
xarray/xarray	20	45.0
pandas-dev/pandas	10	40.0
Overall	**500**	**54.2**

C Compute Requirements

Table 12: Compute requirements for training and inference.

Stage	**GPU Hours (H100)**	**Peak Memory**
SFT (Stage 1)	480	640 GB
RLEF (Stage 2)	960	1.28 TB
Self-improvement ($\times 3$)	960	640 GB
Inference (per task)	0.02	160 GB

Algorithm 1: Reflective Action Cycle (RAC)

```
Input: Task t, Agent a, Memory M, max_steps T, threshold θ,
       patience k
Output: Result r, status s
state ← env.reset(t);
checkpoints ← [state];
stall_count ← 0;
for i ← 1 to T do
    context ← state ⊕ M.retrieve(t);
    thought ← a.reason(context);
    action ← a.act(thought);
    observation ← env.step(action);
    p ← a.critique(state, action, observation);
    M.write(t, thought, action, observation, p);
    if p ≥ θ then
        stall_count ← 0;
        checkpoints.append(state);
    else
        stall_count ← stall_count + 1;
        if stall_count ≥ k then
            state ← checkpoints.pop();
            stall_count ← 0;
        end
    end
    state ← observation;
    if a.is_complete(state, t) then
        return (state, SUCCESS);
    end
end
return (state, TIMEOUT);
```

Table 3: Results on SWE-bench Verified (500 instances). % Resolved indicates the fraction of issues where the generated patch passes all tests.

System	% Resolved	Avg. Steps
SWE-Agent [33]	23.0	42.1
AutoCodeRover [36]	30.7	38.4
Agentless [32]	33.2	12.6
OpenDevin [28]	38.5	51.3
Aider + GPT-4o	41.4	28.7
Devin (commercial)	46.9	–
Zen-Agent (single-agent)	47.3	35.2
Zen-Agent (multi-agent, flat)	50.8	29.6
Zen-Agent (full system)	**54.2**	**26.4**

Table 4: Results on WebArena (812 tasks). Task success requires exact match with the reference answer or successful completion of the specified action sequence.

System	Task Success (%)	Avg. Actions
WebAgent [6]	14.6	22.3
AWM [29]	26.1	19.8
AgentQ [20]	31.4	25.1
SteP [26]	35.8	18.4
Zen-Agent (single-agent)	36.2	21.5
Zen-Agent (multi-agent, flat)	39.7	17.3
Zen-Agent (full system)	**42.8**	**15.9**

Table 5: Results on GAIA by difficulty level. Accuracy is exact-match on the reference answer.

System	Level 1	Level 2	Level 3	Overall
GPT-4 + plugins	68.2	41.3	22.1	48.7
AutoGPT	52.1	33.7	18.4	38.2
HuggingGPT	61.4	38.9	20.7	44.3
FRIDAY	72.3	48.1	31.2	54.1
Zen-Agent (single)	74.1	50.3	34.8	56.2
Zen-Agent (full)	**82.7**	**61.4**	**43.2**	**65.3**

Table 6: Ablation study on SWE-bench Verified (100-instance subset). Each row removes one component while keeping others intact.

Configuration	**% Resolved**	**Δ**
Full Zen-Agent	55.0	–
− HTG Planner (flat decomposition)	50.0	−5.0
− RAC Critique phase	48.0	−7.0
− RAC Backtracking	49.0	−6.0
− Shared Episodic Memory	51.0	−4.0
− Compound tool calls	53.0	−2.0
− RLEF training	46.0	−9.0
− Self-improvement rounds	52.0	−3.0
Single agent (Coder only)	44.0	−11.0

Table 7: Effect of communication topology on SWE-bench (100-instance subset).

Topology	**% Resolved**	**Msg Count**	**Latency (s)**
Star	52.0	142	8.3
Mesh	53.0	387	4.1
Hierarchical	**55.0**	198	5.2

Table 8: Effect of SEM capacity on SWE-bench resolution rate.

SEM Capacity	**% Resolved**	**Retrieval Latency (ms)**
100 records	49.0	0.3
500 records	53.0	0.8
1,000 records	55.0	1.4
5,000 records	55.0	4.7
10,000 records	54.0	12.1

Table 9: Effect of RAC patience parameter k on SWE-bench.

Patience k	**% Resolved**	**Avg. Steps**	**Backtracks**
1	47.0	34.2	18.1
2	51.0	29.8	9.4
3	**55.0**	26.4	5.7
5	53.0	28.1	3.2
8	49.0	31.7	1.8

Table 10: Effect of Orchestrator model scale on SWE-bench Verified.

Orchestrator Model	**% Resolved**	**Cost ($/task)**
Zen-MoDE 7B	38.0	0.42
Zen-MoDE 14B	45.0	0.87
Zen-MoDE 32B	51.0	1.63
Zen-MoDE 72B	**55.0**	3.21